MACHINE LEARNING UNIVERSITY

Building a Multi-Agent AI Chat Application with Kiro

Hands-On Lab Guide - Using Kiro Spec Mode and Strands Agents SDK

Expected duration: ~ 60 minutes | Level: 200 - Intermediate | Version: 1.0

Table of Contents

1. Lab Overview
2. What You Will Build
3. Prerequisites
4. Understanding the Resources and Technologies
5. Part 1 - Setting Up Your Environment
6. Part 2 - Starting with Kiro Spec Mode
7. Part 3 - Requirements Phase
8. Part 4 - Design Phase
9. Part 5 - Implementation (Tasks) Phase
10. Part 6 - Running and Testing Your Application
11. Part 7 - Exploring the Final Application
12. Appendix - Troubleshooting and Glossary

1. Lab Overview

In this hands-on lab, you will use **Kiro**, an AI-powered IDE, to buil. Instead of writing code manually, you will leverage **Kiro's Spec Mode** - a structured workflow that guides you through requirements gathering, system design, and task-based implementation.

By the end of this lab, you will have built a web application where users can chat with an AI assistant that can perform calculations, run Python code, and search for academic papers using a delegated sub-agent architecture.

This lab is part of the **MLU EEP Module 2 - Agentic AI Applications**. The full course material can be found here: https://github.com/aws-mlu/aws-mlu-eep-generative-ai

Learning Objectives

- Understand how to use Kiro's Spec Mode to plan and build software iteratively
- Learn the three phases of Spec Mode: Requirements, Design, and Tasks
- Build a multi-agent system using the Strands Agents SDK
- Create a Flask-based backend with Server-Sent Events (SSE) for real-time status updates
- Understand agent delegation patterns (main agent delegating to specialized sub-agents)
- Deploy and test a complete AI chat application locally

2. What You Will Build

The final application is a **Multi-Agent AI Chat** with the following components:

Key Features

Component	File	Description
Flask Backend	`app.py`	REST API server handling chat messages, agent orchestration, SSE streaming, and session reset
Chat Frontend	`index.html`	Responsive single-page chat UI with real-time agent status indicators
CLI Agent Example	`agent.py`	Standalone script demonstrating multi-turn conversation memory

- **Multi-turn conversations** - The agent remembers context across messages
- **Tool usage** - Calculator, Python REPL, and academic paper search
- **Agent delegation** - Main agent delegates paper searches to a specialized Paper Search Agent
- **Real-time status** - SSE-based streaming shows which agent is working and what it is doing

- **Session management** - Reset button clears conversation history

3. Prerequisites

Before starting this lab, ensure you have the following:

AWS Credentials Note: You need AWS credentials with Amazon Bedrock access. If you are running this lab as an EEP MLU course hands-on, your faculty may provide the proper credentials for running this lab. Make sure they are configured before starting Part 5.

Requirement	Details
Kiro IDE	Installed and configured on your machine. Download from `https://kiro.dev`
Python 3.10+	Verify with `python3 --version`
pip	Python package manager (comes with Python)
AWS Credentials	Configured with access to Amazon Bedrock. Run `aws configure` or set environment variables. Your faculty can help you having the variables needed
Web Browser	Any modern browser (Chrome, Firefox, Safari, Edge)

4. Understanding the Resources and Technologies

Before diving into the lab, let us understand each technology and resource used in this solution.

4.1 Kiro and Spec Mode

Kiro is an AI-powered IDE that helps developers build software through structured workflows. **Spec Mode** is Kiro's guided development approach that breaks down software creation into three phases:

- **Requirements** - Define user stories and acceptance criteria in natural language. Kiro generates structured requirements from your prompt, and you validate/refine them.
- **Design** - Kiro proposes a technical architecture, file structure, data flow, and technology choices based on the approved requirements.
- **Tasks** - Kiro breaks the design into ordered implementation tasks. You execute them one by one, and Kiro generates the code for each task.

4.2 Strands Agents SDK

The **Strands Agents SDK** (`strands-agents`) is a Python framework for building AI agents that can use tools and maintain conversation context. Key concepts:

- **Agent** - The core class that wraps an LLM (Large Language Model) with tool-use capabilities and conversation memory.
- **Tools** - Functions the agent can call to perform actions (calculations, code execution, web requests, etc.).
- **System Prompt** - Instructions that define the agent's personality and behavior.
- **Multi-turn Memory** - Agents automatically remember previous messages in a conversation session.
- **@tool decorator** - Allows you to create custom tools that agents can invoke.

4.3 Strands Agents Tools

The `strands-agents-tools` package provides the following pre-built tools used in this lab:

Tool	Purpose
`calculator`	Performs mathematical calculations
`python_repl`	Executes Python code in a sandboxed REPL environment
`http_request`	Makes HTTP requests to external APIs (used by the Paper Search Agent)

4.4 Flask

Flask is a lightweight Python web framework used here to:

- Serve the HTML frontend via the `/` route
- Handle chat messages via the `/chat` POST endpoint
- Stream real-time agent status via the `/status-stream` SSE endpoint
- Reset conversation state via the `/reset` POST endpoint

4.5 Server-Sent Events (SSE)

SSE is a web technology that allows the server to push real-time updates to the browser over a single HTTP connection. In this app, SSE is used to show which agent is currently working and what it is doing (e.g., "Paper Search Agent: Searching for papers: quantum computing").

4.6 Multi-Agent Architecture

This application uses a **delegation pattern** with two agents:

- **Main Agent** - Handles general chat, calculations, and Python code. When the user asks about academic papers, it delegates to the Paper Search Agent via the `search_papers` custom tool.
- **Paper Search Agent** - A specialized agent that uses the `http_request` tool to query the arXiv API and Semantic Scholar API for academic papers.

4.7 Amazon Bedrock

Amazon Bedrock is the underlying AI service that powers the agents. The Strands SDK connects to Bedrock to access foundation models (like Claude) that understand natural language and can decide when to use tools. Your AWS credentials grant access to this service.

5. Part 1 - Setting Up Your Environment

In this section, you will prepare your local machine for the lab.

Step 1: Create an Empty Project Folder

Open your terminal and create a new empty folder for the project:

```
mkdir mlu-eep-strands-agent-chat

cd mlu-eep-strands-agent-chat
```

Step 2: Open the Folder in Kiro

1. Launch **Kiro** on your computer.
2. Go to **File > Open Folder...**
3. Navigate to and select the `mlu-eep-strands-agent-chat` folder you just created.
4. Kiro will open with an empty workspace.

Step 3: Verify Python Installation

Open Kiro's integrated terminal (**Terminal > New Terminal** or `Ctrl+`) and verify Python is available:

```
python3 --version
```

You should see `Python 3.10.x` or higher.

Step 4: Create and Activate a Python Virtual Environment

This is a **mandatory step**. A virtual environment isolates the lab's dependencies from your system Python installation, keeping your existing setup safe and avoiding conflicts with other projects.

Run the following commands in Kiro's terminal (or ask Kiro to activate a python environment for you):

```
python3 -m venv .venv

source .venv/bin/activate
```

On Windows, use this instead to activate:

```
.venv\Scripts\activate
```

After activation, your terminal prompt should show `(.venv)` at the beginning, confirming you are inside the virtual environment. All packages installed from this point forward will only exist inside this environment.

Important: Make sure you see `(.venv)` in your terminal prompt before continuing. Every time you open a new terminal in Kiro during this lab, you must re-activate the environment with `source .venv/bin/activate`. All `pip install` and `python` commands in this lab assume the virtual environment is active.

Step 5: Install Lab Dependencies

With the virtual environment active, install the required Python packages (it can take some minutes):

```
pip install strands-agents strands-agents-tools flask
```

This installs the three packages needed for the lab. Verify the installation succeeded:

```
pip list | grep -E "strands|flask"
```

You should see `strands-agents`, `strands-agents-tools`, and `flask` in the output.

Step 6: Verify AWS Credentials

Still in the same terminal, verify your AWS credentials are configured:

```
aws sts get-caller-identity
```

You should see your account ID and ARN. If this fails, ask help from your faculty to configure your credentials:

```
aws configure
```

Tip: If your instructor provided temporary credentials, you could set them as environment variables:

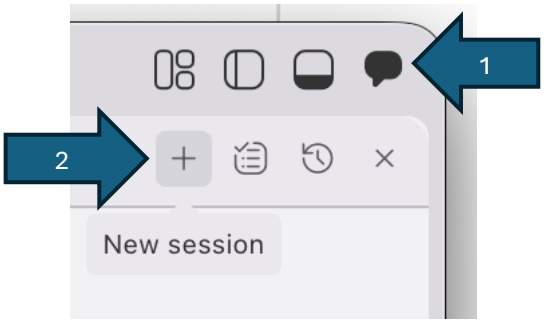
```
export AWS_ACCESS_KEY_ID=your_access_key

export AWS_SECRET_ACCESS_KEY=your_secret_key

export AWS_SESSION_TOKEN=your_session_token

export AWS_DEFAULT_REGION=us-east-1
```

6. Part 2 - Starting with Kiro Spec Mode

Now that your environment is ready, let us use Kiro's Spec Mode to plan and build the application.

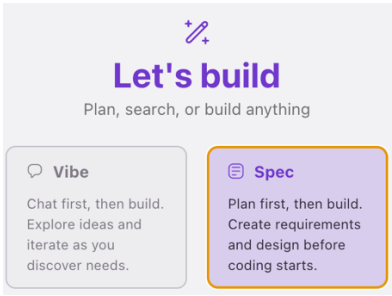
Step 1: Open the Kiro Chat Panel

1. In Kiro, locate the **chat panel** on the right side of the IDE (or open it via the Kiro icon in the right top bar).
2. Select the plus sign and then New Session.



Step 2: Select Spec Mode

1. Click the **"Spec"** box.



2. Kiro will switch to Spec Mode, and you will see a prompt input area where you can describe what you want to build.

What is Spec Mode? Spec Mode is a structured development workflow in Kiro. Instead of writing code directly, you describe what you want to build in natural language. Kiro then guides you through three phases: Requirements, Design, and Tasks. This approach ensures you think through the problem before jumping into code, resulting in better-structured applications.

Step 3: Enter the Prompt

In the Spec Mode input, type the following prompt. This is the description of the application you want to build:

Prompt to Enter in Kiro Spec Mode:

Build a multi-agent AI chat web application using Python. The app should have:

1. A Flask backend (app.py) that serves a chat API with the following endpoints:
- GET / to serve the HTML frontend
- POST /chat to send messages to an AI agent and get responses
- POST /reset to clear conversation history and start fresh
- GET /status-stream as an SSE endpoint that streams real-time agent status updates to the frontend

2. A main AI agent built with the Strands Agents SDK (strands-agents package) that has:
- Calculator tool for math operations
- Python REPL tool for executing code
- A custom search_papers tool that delegates to a specialized Paper Search Agent

3. A Paper Search Agent (sub-agent) that uses the http_request tool from strands-agents-tools to search for academic papers via the arXiv API and Semantic Scholar API.

4. A responsive single-page HTML/CSS/JS chat frontend (index.html) with:
- A clean chat bubble interface with user and agent messages
- Real-time status indicators showing which agent is working (using SSE)
- A reset button to clear the conversation
- Gradient background styling and modern UI

5. A standalone multi-turn conversation example script (agent.py) that demonstrates how the Strands agent maintains context across multiple conversation turns using the calculator tool.

Use strands-agents, strands-agents-tools, and flask as the Python dependencies. The main agent should use a thread-safe queue to push status updates to the SSE stream so the frontend can show real-time progress.

Step 4: Submit the Prompt

Press **Enter** or click the **Send** button to submit your prompt. Kiro will begin processing and will generate the **Requirements** phase.

If Kiro offers options, accept the recommended options: **Build a feature** and then **Start with the Requirements**

Be Patient: Kiro may take 1-2 minutes to generate the requirements. This is normal as it is analyzing your prompt and creating structured user stories.

7. Part 3 - Requirements Phase

After submitting your prompt, Kiro enters the **Requirements Phase**. This is where Kiro translates your natural language description into structured, actionable requirements.

What Happens Automatically

Kiro will generate a **requirements document** (stored as a `.md` file inside the `.kiro/specs/` folder in your project). This document will contain:

- **User Stories** - Written in the format "As a [user], I want [feature], so that [benefit]"
- **Acceptance Criteria** - Specific, testable conditions that must be met for each user story
- **Functional Requirements** - What the system must do

Step 1: Review the Generated Requirements

Once Kiro finishes generating, a requirements file will appear in the Kiro panel. Carefully read through each requirement. You should expect to see user stories like:

- *"As a user, I want to send chat messages and receive AI responses so that I can have conversations with the agent."*
- *"As a user, I want the agent to perform calculations when I ask math questions."*
- *"As a user, I want to search for academic papers through the chat interface."*
- *"As a user, I want to see real-time status updates showing which agent is processing my request."*
- *"As a user, I want to reset the conversation to start fresh."*

Step 2: Validate the Requirements

This is a critical step. Review each requirement and ask yourself:

1. Does this requirement match what I described in my prompt?
2. Are the acceptance criteria specific enough to verify?
3. Is anything missing from my original description?
4. Are there any requirements that seem incorrect or unnecessary?

For this lab you can just accept the requirements created. For a real case, always follow the best practices as below.

Best Practice: Do not just click through the requirements. Actually, read them. This is your chance to catch misunderstandings early, before any code is written. It is much cheaper to fix a requirement than to rewrite code.
This is a key phase for a successful project!
It allows the engineering team to make sure the functional/business features are fully covered and mapped to clearly stated requirements. It is a bridge between the subject

matter experts (SME) - the ones that knows about the problem the system will solve - and the engineering team - the ones that will build the solution. You shouldn't approve the requirements if there is any question or concern on the business expert's side.

Step 3: Request Changes (If needed)

If you notice something missing or incorrect, you can type feedback in the Kiro chat. For example:

- *"Add a requirement for the agent to remember conversation context across multiple messages"*
- *"The paper search should use only arXiv APIs"*
- *"Remove the requirement about user authentication - this app does not need login"*

Kiro will update the requirements based on your feedback.

Step 4: Approve the Requirements

Once you are satisfied that all requirements are correct and complete, follow Kiro's instructions to go further or (or confirm in the chat) to approve the requirements. This tells Kiro to move to the next phase.

Checkpoint: At this point, you should have a .kiro/specs/ folder in your project containing a requirements document. The requirements should cover: chat messaging, calculator tool, Python REPL tool, paper search delegation, SSE status streaming, reset functionality, and the standalone agent example.

8. Part 4 - Design Phase

After approving the requirements, Kiro automatically moves to the **Design Phase**. Here, Kiro creates a technical design document that describes how the application will be built.

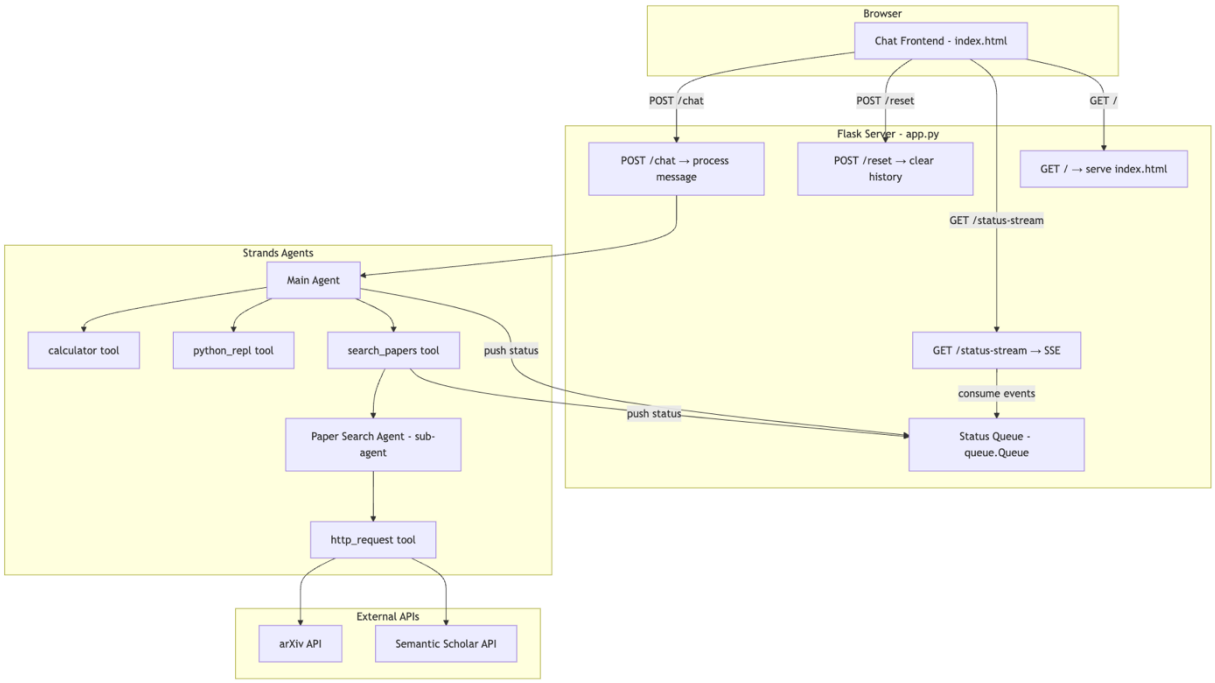
What Happens Automatically

Kiro generates a design document that typically includes:

- **Architecture Overview** - How the components fit together
- **File Structure** - Which files will be created and what each one does
- **Technology Choices** - Libraries, frameworks, and APIs to be used
- **Data Flow** - How data moves through the system (user to frontend to Flask to agent to tools to response)
- **API Design** - Endpoint definitions, request/response formats

Step 1: Review the Architecture

The design should describe an architecture with these components (the architecture may vary but should be similar to this:



Step 2: Verify the Technology Stack

Technology	Purpose	Why This Choice
Flask	Web framework	Lightweight, minimal boilerplate, perfect for API backends
Strands Agents SDK	AI agent framework	Native tool-use support, conversation memory, Amazon Bedrock integration

strands-agents-tools	Pre-built tools	Provides calculator, python_repl, and http_request out of the box
Server-Sent Events	Real-time updates	Simpler than WebSockets for one-way server-to-client streaming
Python threading + queue	Thread-safe status updates	Flask handles requests in threads; queue ensures safe cross-thread communication
Vanilla HTML/CSS/JS	Frontend	No build step needed, simple to serve from Flask

Confirm the design specifies these technologies:

Step 3: Review the API Design

Method	Endpoint	Request Body	Response
GET	`/`	-	Serves `index.html`
POST	`/chat`	`{"message": "string"}`	`{"response": "string"}`
POST	`/reset`	-	`{"status": "reset"}`

GET	`/status-stream`	-	SSE stream of `{"agent": "name", "status": "text"}`

The design should define these endpoints:

Step 4: Approve the Design

If the design looks correct and covers all the components, follow Kiro's instructions to go further or (or confirm in the chat) to approve it and move to the Tasks phase.

Note: For this lab, Kiro may have created everything accordingly, so you can approve it.

Best Practice: If you notice the design is missing the multi-agent delegation pattern (main agent delegating to paper search agent via a custom tool), mention it in the chat before approving. The design should clearly show two separate Agent instances and a `@tool`-decorated function bridging them.
Checkpoint: Your `.kiro/specs/` folder should now contain both a requirements document and a design document. The design should describe the Flask backend, two Strands agents, SSE streaming, and the HTML frontend.

9. Part 5 - Implementation (Tasks) Phase

This is where the code gets written. After approving the design, Kiro generates an ordered list of **implementation tasks**. Each task is a discrete unit of work that builds on the previous ones.

Understanding the Task List

The tasks may be different every time you run this lab, even using the same initial prompt, but it should contain tasks related to the content related to the key elements being implemented:

1. Multi-Agent Architecture

- A Main Agent equipped with calculator, Python REPL, and custom search_papers tools
- A transient Paper Search Agent (sub-agent) that queries arXiv and Semantic Scholar APIs
- Thread-safe status queue for real-time agent activity updates

2. Flask Web Application

- GET / endpoint serving the chat frontend
- POST /chat endpoint for processing user messages with conversation history
- POST /reset endpoint to clear conversation state

- GET /status-stream Server-Sent Events (SSE) endpoint for real-time status updates

3. Chat Frontend (HTML/CSS/JavaScript)

- Modern UI with gradient background and responsive design
- Distinct user/agent message bubbles
- Real-time status indicator showing active agent names
- Auto-scrolling and reset functionality

4. Testing Strategy

- Property-based tests using Hypothesis for universal correctness (5 properties covering response structure, conversation history, reset behavior, status lifecycle, and sub-agent status)
- Unit tests for endpoint error handling and edge cases
- Mocked agent calls to avoid LLM dependency

5. Standalone Demo Script

- agent.py demonstrating multi-turn conversations with context persistence

The implementation follows an incremental approach with checkpoints, starting from project setup through core agent logic, Flask endpoints, frontend, and finally the standalone script.

How to Execute Tasks

For each task in the list, follow this workflow:

Step A: Read the Task Description

Click on the task to expand its details. Kiro provides a description of what needs to be done, including which files to create or modify.

Step B: Click "Start Task"

For each task, click on the **"Start Task"** link on the task.md document to tell Kiro to begin implementing this task. Kiro will generate the code and create/modify the necessary files.

Note: sometimes the document doesn't render properly, and the link is not available. In this case, asking Kiro to "run task #" in the chat window also works.

Step C: Review the Generated Code

After Kiro finishes a task, review the code it generated. Open the created/modified files and check:

- Does the code match the task description?
- Are there any obvious errors or missing pieces?
- Does it follow good coding practices?

Step D: Provide Feedback (If needed)

If the generated code needs changes, type your feedback in the Kiro chat. For example:

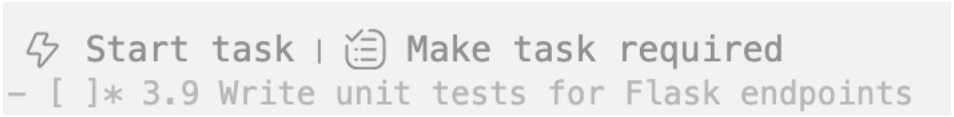
- *"The SSE endpoint should use a 30-second timeout for the queue.get() call"*
- *"Add error handling for when the agent call fails"*

Step E: Mark Task as Complete

Once you are satisfied with the implementation, the task will be marked as complete, and you can move to the next one.

Important: Execute tasks **in order**. Each task may depend on files or code created by previous tasks. Skipping ahead can cause errors.
Kiro use to create test sub-tasks when needed. In case Kiro says that some test error is not important to move further, make sure to check and agree with it before approving it.

Note: some tasks are by default marked as optional by default by Kiro in order to speed up the process for an MVP first delivery. Those tasks appear as light gray similer to the figure below:



It is ok to leave them as not required if you want to have a faster result. Usually, Kiro creates a working app for your tests. However, for a final deployment, it is recommended that all not required tasks be marked as required. You can do it by just click the link offered by the text “Make task required” or by removing the asterisk befor the name of the task in the tasks.md file.

Checkpoint: After completing all tasks, your project folder should contain three files: app.py, agent.py, and template/index.html. The .kiro/specs/ folder should contain the requirements, design, and tasks documents.

10. Part 6 - Running and Testing Your Application

Now that all the code is generated, let us run and test the application.

Step 1: Confirm Your Virtual Environment Is Active

Check that your terminal prompt shows (.venv). If it does not, re-activate the virtual environment you created in Part 1:

```
source .venv/bin/activate
```

Step 2: Verify AWS Credentials

```
aws sts get-caller-identity
```

If this fails, configure your credentials before proceeding.

Step 3: Run the Application

In Kiro's terminal, start the Flask server:

```
python app.py
```

You should see output like:

```
 * Serving Flask app 'app'
 * Debug mode: on
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Step 4: Open the Chat Interface

Open your web browser and navigate to:

```
http://localhost:5000
```

Note: the 5000 port may change. Follow Kiro instructions about the right port.

You should see the chat interface with a gradient purple background and a welcome message from the Main Agent.

Step 5: Test the Features

Try each of these test scenarios to verify everything works:
Note: if the output is not what you expect, try asking Kiro to fix it!

#	Test	What to Type	Expected Behavior
1	Basic chat	*"Hello! What can you do?"*	Agent responds describing its capabilities
2	Calculator	*"What is 42 * 17 + 3?"*	Agent uses calculator tool and returns 717
3	Python REPL	*"Write a Python function to check if a number is prime and test it with 17"*	Agent writes and executes Python code
4	Memory	*"My name is Alice"* then *"What's my name?"*	Agent remembers your name across messages
5	Paper search	*"Find papers about transformer neural networks"*	Status shows Paper Search Agent working, then returns paper results
6	Reset	Click **"Reset Chat"** then ask *"What's my name?"*	Agent no longer remembers your name

What to watch for during paper search: When you ask about papers, watch the chat area for real-time status bubbles. You should see:

1. A status bubble like: *"Main Agent: Processing your request..."*
2. A different status bubble: *"Paper Search Agent: Searching for papers: transformer neural networks"*
3. The final response appears as a regular chat message

Step 6: Test the Standalone Agent (Optional)

Open a new terminal tab and run the standalone example:

Note: if you are using a temporary credential and have access error, refresh your credentials and try again (If you need assistance, your faculty can help you).

```
python agent.py
```

This will run 4 conversation turns automatically, demonstrating that the agent remembers context (name, favorite number, and/or previous calculation results).

Checkpoint: If all test scenarios pass, congratulations - your multi-agent chat application is fully functional!

11. Part 7 - Exploring the Final Application

Now that the application is running, let us explore the code to understand how everything fits together.

11.1 The Agent Delegation Pattern

Ask Kiro: Explain the code of the search_papers function in app.py. What are the key observations?

Key observations:

- The `@tool` decorator from Strands makes this function available to the main agent as a tool
- The docstring is critical - the LLM reads it to decide when to use this tool
- Inside, it calls the `paper_search_agent` directly, which is a completely separate Agent instance
- Status updates are sent before and after the sub-agent call

11.2 The SSE Streaming Mechanism

Ask Kiro: explain in a short text the Streaming mechanism implemented here

11.3 The Reset Mechanism

Ask Kiro: explain in a short text the Reset mechanism implemented here

11.4 Frontend Architecture

Ask Kiro: explain in a short text the frontend architecture implemented here

12. Appendix

A. Complete File Summary

File	Purpose
app.py	Flask backend with Main Agent, Paper Search Agent, SSE streaming, and all API endpoints
index.html	Single-page chat frontend with CSS styling, JS logic, and SSE client
agent.py	Standalone multi-turn conversation demo script

B. Python Package Dependencies

Package	PyPI Name	What It Provides
Strands Agents SDK	`strands-agents`	Core `Agent` class, `@tool` decorator, conversation memory, Bedrock integration
Strands Agents Tools	`strands-agents-tools`	Pre-built tools: `calculator`, `python_repl`, `http_request`
Flask	`flask`	Web framework: routing, request handling, response streaming

C. External APIs Used by the Paper Search Agent

API	Endpoint	Purpose
arXiv API	http://export.arxiv.org/api/query	Search for academic papers on arXiv (open access preprint repository)
Semantic Scholar API	https://api.semanticscholar.org/graph/v1/paper/search	Search for academic papers across multiple sources with citation data

D. Troubleshooting

Problem	Cause	Solution
`ModuleNotFoundError: No module named 'strands'`	Dependencies not installed	Run `pip install strands-agents strands-agents-tools flask`
`botocore.exceptions.NoCredentialsError`	AWS credentials not configured	Run `aws configure` or set environment variables
`AccessDeniedException` from Bedrock	AWS account does not have Bedrock access	Request Bedrock model access in the AWS Console, or check your region
Chat sends message but no response	Agent call is taking too long	Check the terminal for errors. The first call may take 10-20 seconds due to cold start
Status bubbles do not appear	SSE connection failed	Check browser console for errors. Ensure the app runs with `threaded=True`
Paper search returns no results	External API may be down or rate-limited	Try a different search query, or wait a moment and retry
`Address already in use`	Port 5000 is occupied	Kill the existing process or change the port in `app.py`

Kiro Spec Mode not generating tasks	Prompt may be too vague	Use the detailed prompt provided in Part 2, Step 3 of this guide

E. Kiro Spec Mode Best Practices Summary

1. **Write detailed prompts** - The more specific your initial prompt, the better the generated requirements, design, and code will be. Include technology choices, API endpoints, and UI details.
2. **Actually read the requirements** - Do not skip the review step. Catching a wrong requirement early saves hours of rework later.
3. **Validate the design before proceeding** - Make sure the architecture makes sense and all components are accounted for before moving to tasks.
4. **Execute tasks in order** - Tasks are ordered by dependency. Skipping ahead can cause missing imports or undefined references.
5. **Review generated code** - Kiro generates good code, but always review it. Check for edge cases, error handling, and security considerations.
6. **Use the chat for feedback** - If something is not right, tell Kiro in the chat. It can modify requirements, design, or code based on your feedback.
7. **Test incrementally** - After each major task, try running the code to catch issues early rather than waiting until the end.
8. **Iterate** - Spec Mode is iterative. You can go back and refine requirements or design if you discover something during implementation.

F. Glossary

Term	Definition
Agent	An AI entity that can understand natural language, use tools, and maintain conversation context
Tool	A function that an agent can call to perform a specific action (e.g., calculate, search, execute code)
Multi-turn conversation	A conversation where the AI remembers what was said in previous messages
Agent delegation	Pattern where one agent passes a task to another specialized agent
SSE (Server-Sent Events)	A web standard for servers to push real-time updates to browsers over HTTP
LLM (Large Language Model)	The AI model that powers the agent's understanding and responses
Amazon Bedrock	AWS service providing access to foundation models for building AI applications
Flask	A lightweight Python web framework for building web applications and APIs
Spec Mode	Kiro's structured development workflow: Requirements, Design, Tasks

@tool decorator	A Python decorator from Strands SDK that registers a function as a tool an agent can use